FPL Automation Framework Project Reference

Architecture, agents, execution workflows and operational guidance

NAMS, NEO and the multi application automation platform

Prepared 26 October 2026 | README reference edition

What this framework is

The framework combines governed AI assisted test authoring with repeatable browser test execution. Its control plane routes work to application agents and lifecycle skills; Playwright and Cucumber execute the resulting tests and produce evidence for reviewers. These are related but distinct responsibilities.

What this reference provides

Use this document to orient a new engineer, explain the system to a reviewer, locate the correct project layer, select a test run, and interpret its reports. It includes reconstructed architecture diagrams, agent responsibilities, lifecycle and execution swimlanes, setup commands, application mappings and contribution rules.

Important boundaries

The architecture and commands describe the documented project, not a certification of a running deployment. App readiness differs: some commands are conditional on missing or rewritten specs, and Fuels is explicitly a scaffold. The evidence register records the README screenshot coverage and the files needed for implementation level verification.

How to read the evidence

- Documented: directly supported by the visible README, referenced as S01 to S13.
- Explanation: interpretation that makes the documented design easier to understand; not an additional implementation claim.
- Recommendation: a proposed review or operating practice, not evidence that a control is already configured.

Contents and reading paths

Section	Page	Use
Project purpose and people	3	Business scope and responsibilities
System architecture	4	Authoring and execution boundaries
Layered architecture	5	Routing and ownership
Agents and lifecycle skills	6	Responsibility matrix
Authoring swimlane	7	From requirement to reviewed change
Runtime swimlane	8	From launch to evidence
Application runtime map	9	Specs, POMs and fixtures
Tools and versions	10	Documented technology baseline
Repository navigation	11	Where files belong
Local setup	12	PowerShell walkthrough
Configuration and selection	13	APP_ID, TEST_ENV and SUITE_ID
Application run examples	14	Commands and readiness
Reporting and QMetry	15	Results and report commands
AI tooling and MCP	16	IDE agents and integrations
Branches and standards	17	Review and promotion
Security and reliability	18	Controls to verify
Troubleshooting and handoff	19	Operational checklists
Evidence and open questions	20	Source inventory and gaps

Choose your starting point

New engineer: read pages 3 to 6, then 11 to 14. Reviewer or architect: read pages 4 to 8, then 17 to 20. Test operator: start at pages 12 to 15 and use page 19 when a run does not behave as expected.

Commands are examples, not instructions that have been executed for this document. Values in angle brackets are placeholders. Use the authorized repository URL, test accounts and target environments.

Project purpose and people

Evidence: S01 to S03

The README describes automated testing for FPL nuclear plant management applications. NAMS covers asset management; NEO covers planning, scheduling and engineering; LaunchPad is also named in the overview. The broader runtime map includes nine application identifiers.

The browser testing loop

A test opens a browser, navigates to an application, signs in as a test user, and performs actions such as opening work orders, updating fields and verifying data. Assertions check expected behavior. The run produces pass or fail results with screenshots, videos and traces. The README describes local development runs and CI/CD execution on changes or schedules.

Explanation: these are application workflow tests. Nothing in the source establishes control of plant equipment, safety certification, comprehensive business coverage or a guarantee that a release is defect free.

Participant	Documented responsibility
QA engineer	Write and maintain tests; inspect results.
Developer	Run tests locally to check that code changes do not break workflows.
QA lead	Review feature PRs into weekly-integration; approve promotion to dev; monitor test health.
Project manager	Use Allure reports to review coverage and pass rates.
CI/CD pipeline	Execute tests automatically on schedule or per commit.

How to judge a useful result

Recommendation: a report should identify the selected application, environment, test cases, code revision and execution outcome. A passing test is meaningful only for the behavior and data it actually exercised. A discovery command is useful for checking configuration but is not a passing browser test.

Where business knowledge lives

The README places knowledge bases, QMetry or manual references and handoffs under app-context/<app>/. Application agents and skills use that context. The contents and quality of those references must be checked before deriving a new test.

System architecture

Evidence: S02, S06 to S10, S13

The system has an authoring path that creates and repairs tests, and an execution path that runs them. They meet at the repository artifacts. The diagram is an explanatory reconstruction of the README, not a deployment topology.

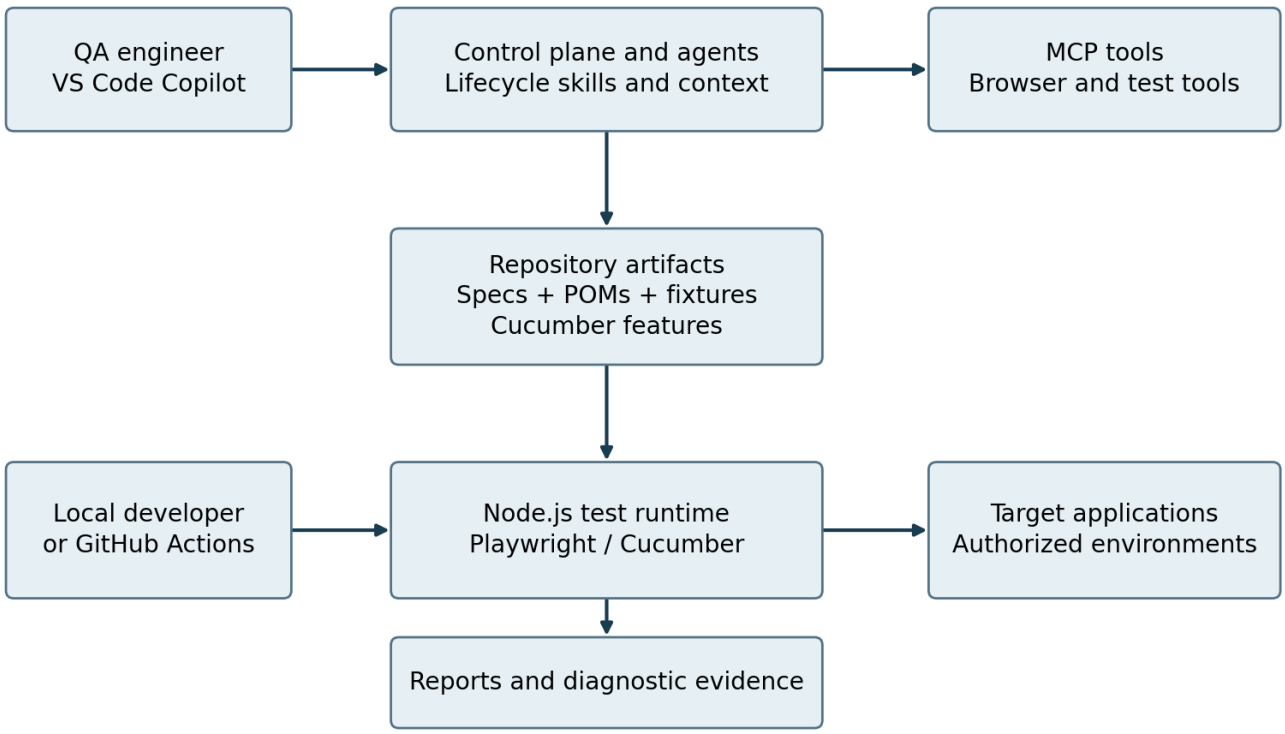


Figure 1. Logical system context. MCP is an authoring tool interface; it is not shown as a required intermediary for every test run.

Boundaries that matter

- Authoring: routing rules determine the allowed phase before tools or app changes are used.
- Runtime: a runner executes specs, POMs, fixtures or Cucumber features against the selected app.
- Evidence: reporters produce artifacts; JUnit output is used by the documented QMetry upload path.
- Review: human approval and the branch promotion path remain distinct from AI generation.

The README does not establish exact network paths, infrastructure hosting, service accounts, pipeline YAML, application URLs or live QMetry upload status. Those require direct configuration evidence.

Layered architecture and ownership

Evidence: S06 to S09

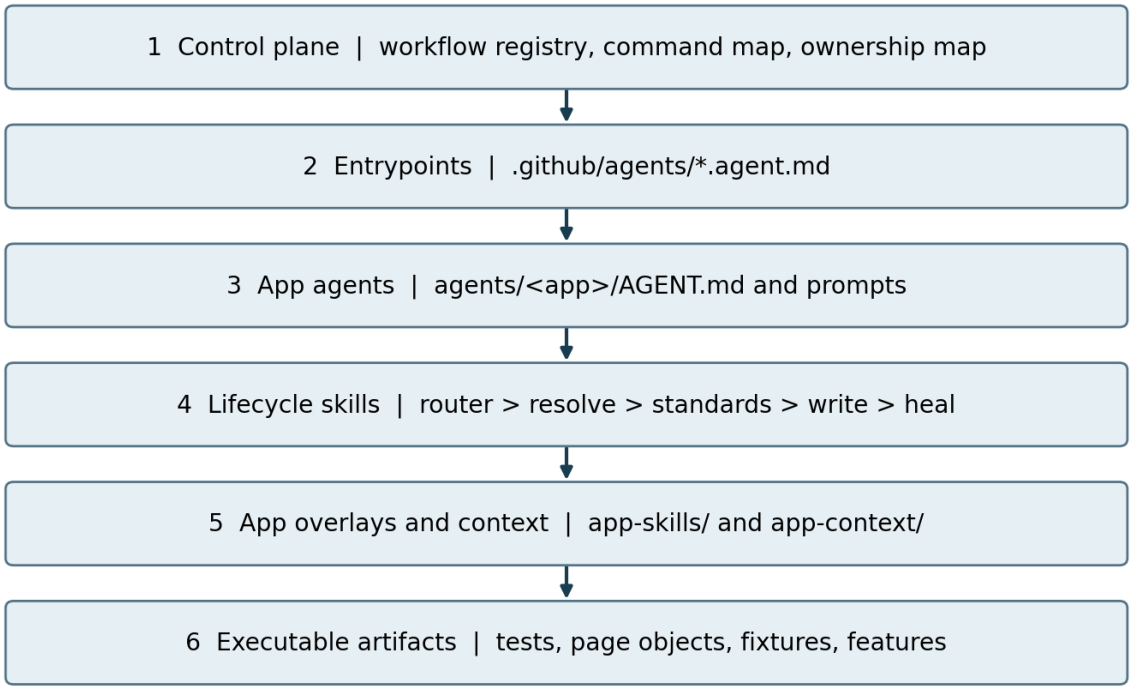


Figure 2. Reader oriented layer map. The bottom layer is executed by runners; the upper layers govern creation and maintenance.

Documented precedence is control plane routing, agent, skill, app overlay and app context knowledge. Read .github/control-plane/README.md and AGENTS.md for entry rules. The README explicitly forbids freestyle POM edits for lifecycle apps before the router assigns a phase.

Platform skills are a protected shared layer

.github/skills/ contains app agnostic methods such as MCP generation, API, JMeter, Jira and migration helpers. App teams must not edit this layer to implement app behavior. They extend behavior through lifecycle skills, application overlays and app context instead.

Routing is not execution

Explanation: the control plane answers which workflow and owner apply. Skills supply the procedure, context supplies app knowledge, and executable files supply runtime behavior. A diagram arrow indicates a logical relationship; it does not prove a particular API call or process boundary.

Agent and skill responsibilities

Evidence: S07 to S11

The README names two AI authoring concepts: app lifecycle agents and Playwright IDE agents. Do not treat them as a single autonomous runtime service.

Component	Responsibility	Boundary
Control plane	Choose workflow, ownership and approval gates.	Routing only; consult entry rules.
App lifecycle agent	Enter the correct app workflow through AGENT.md and prompts.	App scope; routes to lifecycle skills.
Router skill	Assign the lifecycle phase.	Required before app POM edits or tool use.
Resolve and standards	Resolve context and apply standards in the phased chain.	Detailed internal contracts not supplied.
Write and heal	Create or repair app test artifacts.	Changes still need execution evidence and review.
KB, defects, closeout	Supporting lifecycle capabilities named by README.	Exact outputs and gate criteria need skill files.
Playwright Planner	Create a test plan, for example a Change Request plan.	IDE authoring role in VS Code Copilot.
Playwright Generator	Generate tests from a plan such as specs/my-plan.md.	Not proof generated tests are correct.
Playwright Healer	Help fix a failing test, such as npa-tc-1262.	Not permission to hide product defects.
Platform skills	Shared MCP, API, JMeter, Jira and migration methods.	Do not modify for app behavior.

Application entrypoints visible in the tree

agents/ includes aic, caps-screening, neo-planner-scheduler, neo-work-request-screening, nuclear-work-id and dwp-planner-scheduler. Lifecycle skill prefixes include aic-*, caps-*, dwp-*, neo-*, nwi-* and nwrs-*. An entrypoint name is not evidence that every associated test is complete.

Explanation: the brief descriptions of resolve, standards and support phases clarify their names. Detailed inputs, output schemas and stop conditions must come from the actual skill files. Devin is not established as a framework integration by this README evidence.

Authoring lifecycle swimlane

Evidence: S07 to S12; explanatory handoffs

Read downward through the numbered steps. The lane arrangement explains ownership; it is not a claim that the README implements these steps as an automated workflow engine.

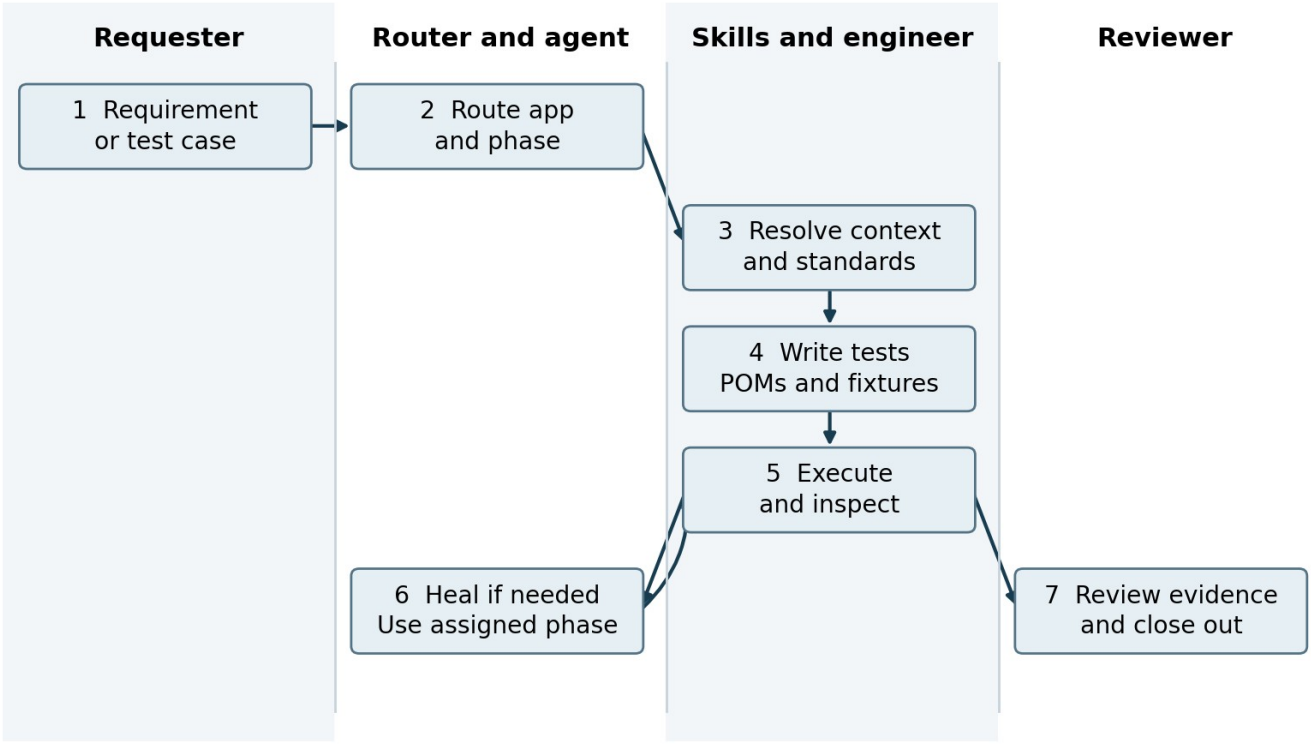


Figure 3. Lifecycle swimlane. Execution and human review are explanatory handoffs added around the documented router to heal chain.

Inputs and handoffs

A requirement or test case identifies the app and expected behavior. The router assigns a phase; the app agent, lifecycle skills, overlays and app context constrain the work. The write phase produces changes in the app test and page object paths. Execution produces diagnostic evidence for repair or review.

Failure is a decision point

Recommendation: classify a failure before healing. It may be an application defect, a test defect, missing data, authentication failure or environment outage. Preserve the evidence and correct the right cause. Do not remove meaningful assertions merely to obtain a pass.

Completion is more than generated code

Recommendation: hand off a diff, selected tests, command and environment, run results, unresolved defects and any context updates. Apply the documented peer review and weekly-integration promotion path before the change reaches dev.

Test execution swimlane

Evidence: S02 to S06, S09 and S13

This flow describes test execution separately from AI authoring. A local engineer or CI/CD pipeline launches a runner; application fixtures and page objects support browser interactions.

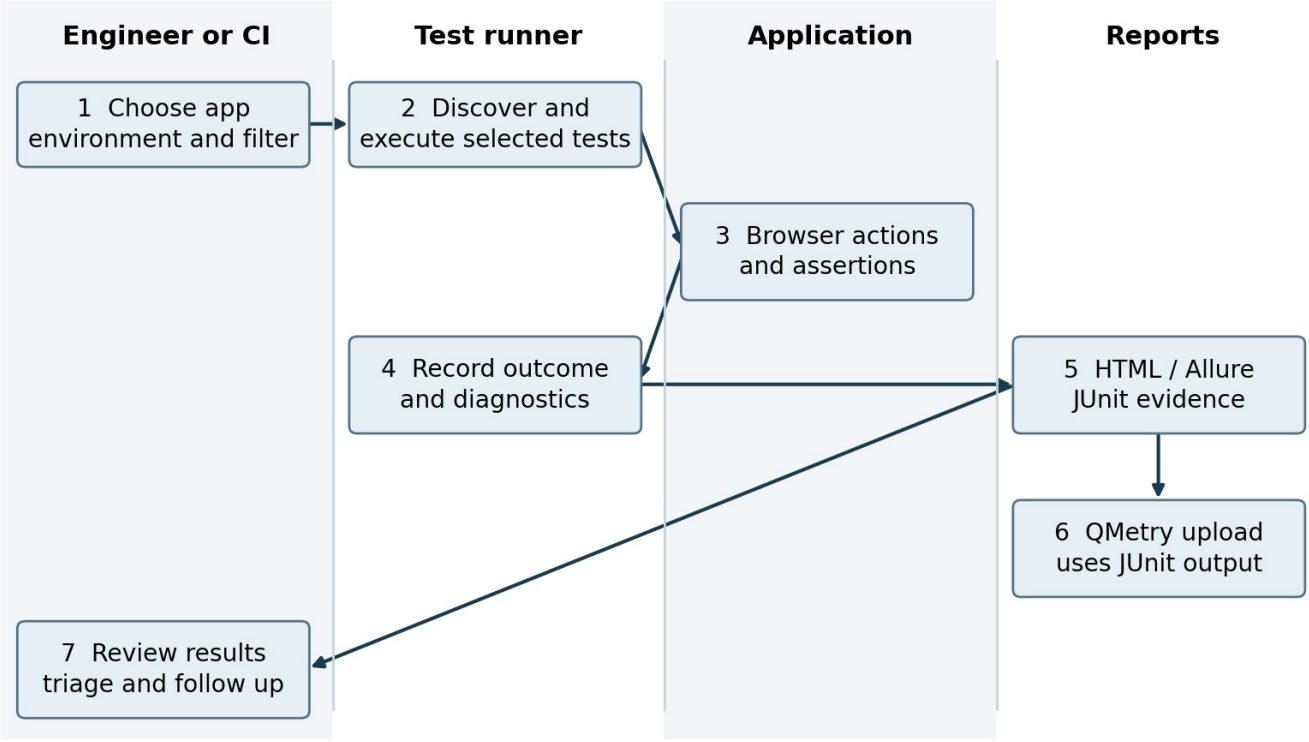


Figure 4. Runtime and evidence swimlane. QMetry upload is a documented downstream use, not a verified successful integration.

Selection and environment are independent

APP_ID chooses the application; TEST_ENV chooses dev, test or qa. Playwright file arguments or --grep select tests, while Cucumber uses --tags. SUITE_ID affects report naming only. It does not turn an unrestricted run into a smoke or regression subset.

Evidence produced

The README describes screenshots, videos and traces alongside pass or fail reports. Reports are stored under test-artifacts/reports/. Every Playwright run is documented to produce a <runStem>.qmetry.xml JUnit file for QMetry upload. Exact capture policies, retention and upload triggers require config and script inspection.

Application runtime map

Evidence: S04 and S09

Common Playwright layout: tests/nuclear/<app>/ for specs and src/pages/<app>/ for POMs. The table gives the app segment and documented exceptions so paths remain readable.

APP_ID	Spec and POM location	Typical fixture or hook
neo	Common layout with app neo.	fixtures/test-fixtures.ts
aic	Common layout with app aic.	fixtures/aic.fixtures.ts
launch-pad	Common layout with app launch-pad.	fixtures/launch-pad.fixtures.ts
nuclear-work-id	Common layout with app nuclear-work-id.	fixtures/nuclear-work-id.fixtures.ts
caps-screening	Common layout with app caps-screening.	fixtures/caps-screening.fixtures.ts
ops	Specs: tests/nuclear/ops/ POM: src/pages/neo/ shared Operability Screening POM.	fixtures/ops.fixtures.ts
fuels	Common layout with app fuels.	Deferred until Fuels authentication is established.
nuas-dwp	Common layout with app nuas-dwp.	src/fixtures/nuas-dwp/test-fixtures.ts
nams	Cucumber features/ plus agentic pages; POM: src/pages/nams/.	Cucumber world and hooks.

Readiness is not uniform

- CAPS Screening and OPS examples depend on an OS-TC spec being added.
- AIC examples are described as usable after specs are rewritten.
- NUAS DWP example demonstrates discovery with --list, not a successful UI run.
- Fuels is scaffold configuration; no UI smoke spec until the authentication and UI contract is provided.

Explanation: a path in this map establishes intended organization, not coverage completeness or operational readiness. Use app documentation and an actual discovery/run result to confirm what exists in a particular checkout.

Tools and documented versions

Evidence: S10 and S13

This is the README baseline, not a recommendation to upgrade or a claim about current public releases. The README says versions come from package.json and package-lock.json and that its version table should be updated in the same PR as an upgrade. Those files were not independently inspected.

Tool	README version	Role
Node.js	24 LTS, CI pin	JavaScript runtime; README directs use of 24 even though engines allows >=20.
Playwright and @playwright/test	1.63.0 exact pin	Browser automation, assertions and parallel execution.
@cucumber/cucumber	9.x	NAMS BDD feature runner.
TypeScript / ts-node	5.4 / 10.9	Typed test code and TypeScript execution.
Allure 3 CLI	3.6	Stakeholder HTML reporting.
allure-playwright / allure-cucumberjs	3.3 / 2.15	Write results for the respective runners.
Playwright HTML and JUnit	Built in	Local reports and QMetry XML output.
xlsx / exceljs / pdf-parse	@e965/xlsx 0.20.3 fork; 4.4; 1.1	Parse downloaded exports in process.
ESLint / TypeScript ESLint / Prettier	8.49 / 5.62 / 2.8	Lint and formatting tools.
Chromium	Bundled with Playwright 1.63	Default test browser.
Playwright Test MCP	Bundled with 1.63	Support Planner, Generator and Healer tooling.
Playwright browser MCP	@playwright/mcp@latest	Interactive browser exploration; separately versioned.
GitHub Actions	Node 24 execution noted	Automated pipeline execution.

Recommendation: confirm the lockfile, runtime version and browser installation together when reproducing a run. The floating browser MCP reference can change independently of the pinned test runner; record the resolved version during troubleshooting.

Repository navigation and file placement

Evidence: S10 to S12

Path	Documented purpose
.github/control-plane/	Workflow registry, command map and ownership map.
.github/agents/	Entrypoint manifests and Playwright IDE agents.
.github/skills/	Shared platform methods; not app-team customization.
agents/	App lifecycle entrypoints with AGENT.md and prompts.
skills/	App lifecycle router, resolve, standards, write, heal and related skills.
app-skills/	Optional app overlays and manifests.
app-context/	Knowledge bases, reference workbooks and handoffs.
addenda/	Skill addenda by application surface.
tests/nuclear/<app>/	Playwright test specs, optionally grouped by functional area.
src/pages/<app>/	Application page objects.
src/support/, src/utils/, src/steps/	Shared support, utilities and Cucumber steps.
fixtures/ and features/	App fixtures; NAMS legacy and agentic Cucumber features.
data/ and config/	Profiles, QMetry maps and app registry.
docs/ and scripts/	Architecture, ledgers, standards; reports, upload and deploy helpers.

Placement rules

New Playwright tests belong under tests/nuclear/<app>/; do not introduce a new top level directory under tests/. Page objects belong under src/pages/<app>/. Specs import the app fixture rather than raw @playwright/test, unless the app documentation explicitly says otherwise.

Read before editing

AGENTS.md contains app scoped agent instructions. CODING_STANDARDS.md is the framework contract, including the branch pathway. .github/CONTRIBUTING.md describes contributions and promotion; .github/pull_request_template.md is the PR template. This reference identifies those documents but does not substitute for their full contents.

Local setup in PowerShell

Evidence: S02, S03 and S13

Run these steps only on an authorized development computer with access to the FPL network and the repository. Use a PowerShell terminal in VS Code or Windows Terminal. This walkthrough normalizes the README examples to one command per line.

1 Verify prerequisites

```
node --version
npm --version
git --version
```

The README baseline is Node.js 24 LTS plus npm and Git. These version commands are supplementary diagnostic checks. Resolve missing tools or network access before installing project dependencies.

2 Clone and install

```
git clone <repo-url>
cd yt213-agentx-automation
npm ci
npx playwright install
```

Replace <repo-url> with the approved repository URL. npm ci uses the project lockfile. Browser installation is a separate step from installing Node dependencies; no install was performed for this reference.

3 Configure a local environment

```
Copy-Item .env.example .env
```

Edit .env with the approved test credentials and target environment values required by the project. Copy-Item is the explicit PowerShell form of the README copy command. Do not paste secrets into chat, screenshots, commits or shared documentation.

4 Run the README example

```
$env:APP_ID="neo"
$env:TEST_ENV="qa"
npx playwright test npa-tc-1262 --reporter=list
```

Use these settings in the same terminal session as the test. PowerShell uses $env:NAME="value"; Command Prompt uses a different syntax. If you were previously using CMD, open PowerShell rather than pasting PowerShell variables into CMD.

Recommendation: first confirm the branch, approved target and test data. A test can perform application actions; do not casually point it at a production environment.

Configuration and test selection

Evidence: S03 to S05

Control	What it changes	What it does not establish
APP_ID	Selected application, such as neo, nams or launch-pad.	Does not prove that app has executable or complete coverage.
TEST_ENV	Environment selection: dev, test or qa.	Does not grant network or application access.
SUITE_ID	Report naming.	Does not filter or select tests.
Playwright --grep	Selects tests matching the supplied pattern.	A report label does not replace this filter.
Cucumber --tags	Selects scenarios matching a tag expression.	Not interchangeable with Playwright --grep.
Playwright --list	Lists discovered tests without executing them.	Not proof of login, browser actions or passing assertions.

NEO regression examples

```
$env:APP_ID="neo"
$env:TEST_ENV="qa"
$env:SUITE_ID="regression"
npx playwright test --grep "regression.*Planner"
npx playwright test --grep "regression.*Scheduler"
```

The two test commands are separate runs. The regular expressions, not SUITE_ID, narrow selection to the documented Planner or Scheduler titles.

NAMS smoke example

```
$env:APP_ID="nams"
$env:TEST_ENV="qa"
$env:SUITE_ID="smoke"
npx cucumber-js --tags "@smoke"
```

Before pressing Enter

Recommendation: confirm the app, environment and selected tests; check that the current shell variables are intentional. A previous value can persist in a terminal. Record the selection in the handoff so another engineer can reproduce the same scope.

Application command examples

Evidence: S03 and S04

Set TEST_ENV to qa for the examples below unless stated otherwise. Each app block sets APP_ID explicitly. These are README examples, not verified run results; retain the conditions shown.

```
$env:TEST_ENV="qa"
```

NEO and NAMS

```
$env:APP_ID="neo"
npx playwright test npatc-853
$env:APP_ID="nams"
npx playwright test --grep "INM-TC-193"
npx cucumber-js --tags "@INM-TC-193"
```

CAPS Screening and OPS after the OS TC spec is added

```
$env:APP_ID="caps-screening"
npx playwright test --grep "OS-TC-914"
$env:APP_ID="ops"
npx playwright test --grep "OS-TC-170"
```

Nuclear Work ID and LaunchPad

```
$env:APP_ID="nuclear-work-id"
npx playwright test nwi-tc-15 --reporter=list
$env:APP_ID="launch-pad"
npx playwright test nh-tc-80 --reporter=list
```

AIC after specs are rewritten

```
$env:APP_ID="aic"
npx playwright test vbm-tc-3400 --reporter=list
npx playwright test vbm-tc-3421 --reporter=list
```

Discovery only for the documented DWP and Fuels examples

```
$env:APP_ID="nuas-dwp"
npx playwright test dwp-tc-98 --list
$env:APP_ID="fuels"
$env:TEST_ENV="dev"
npx playwright test --list
```

Fuels remains a scaffold until its authentication and UI contract is supplied. Do not reinterpret discovery output as a UI smoke result.

Reports and QMetry evidence flow

Evidence: S05, S06 and S13

Allure is the README recommended report for stakeholders. Playwright HTML provides a runner specific view, and NAMS has a Cucumber HTML route. Report files are stored under test-artifacts/reports/.

Command	Documented effect
npm run allure	Generate and open an Allure report.
npm run allure:report	Generate only.
npm run allure:open	Open the last generated report.
npm run report	Open the newest Cucumber HTML report under test-artifacts/reports/.
npm run allure:clear	Delete all results and reports. Destructive cleanup, not a normal viewing step.

Open a specific Playwright report

```
npx playwright show-report "test-artifacts/reports/<runStem>-playwright"
```

Replace <runStem> with the actual run stem; the angle brackets are not a literal directory name. The exact naming algorithm is not visible in the screenshots.

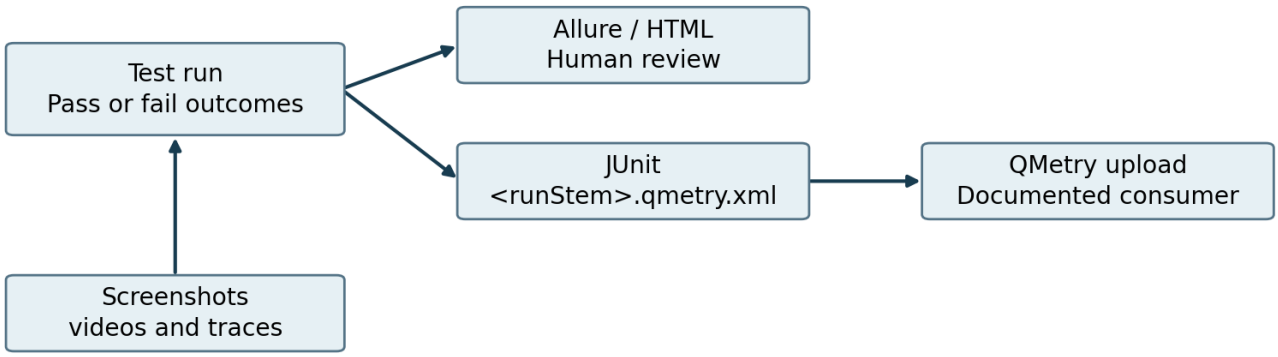


Figure 5. Reporting relationships. No successful QMetry upload or API credentials were validated.

Recommendation: inspect the failure message together with the trace and screenshot, then correlate it to the test ID, app and environment. Preserve evidence before cleanup. Confirm script behavior and retention policy before using allure:clear.

AI tooling and MCP configuration

Evidence: S10 and S13

App lifecycle workflows govern how an application test case is generated, healed and closed out. Separately, Playwright IDE agents in .github/agents/playwright-test-*.agent.md provide Planner, Generator and Healer capabilities inside VS Code Copilot with MCP tools.

Documented IDE prompts

```
@playwright-test-healer Fix the failing npa-tc-1262 test
@playwright-test-planner Create a test plan for the Change Request module
@playwright-test-generator Generate tests from specs/my-plan.md
```

These prompts illustrate the README interface. Their presence does not demonstrate that a tool is connected, that the user has access, or that a generated result has passed review. On lifecycle apps, apply the router assigned phase before platform tool use.

Configuration surface	Documented registration
.mcp.json	Browser MCP uses @playwright/mcp@latest. It is versioned separately from Playwright.
.vscode/mcp.example.json	Template for QMetry MCP using @smartbear/mcp and Playwright Test MCP.
Playwright Test MCP command	npx playwright run-test-mcp-server; ships with @playwright/test 1.63.
.vscode/mcp.json	Copy the example, supply the QMetry API key locally; README explicitly says not to commit this file.
GitHub Copilot Coding Agent	Settings > Copilot > Coding agent > MCP configuration.

Different tools have different trust and access boundaries

Explanation: browser MCP supports interactive exploration; Test MCP exposes test tooling; QMetry MCP connects to test management. These roles should not be conflated with the test runner itself or with a successful report upload. MCP registration alone does not grant permissions.

Controls to verify before use

Recommendation: verify approved servers, access scopes, credential storage, allowed target environments and logs. Review generated diffs and test evidence before merging. The README screenshots do not establish the model provider, model version, costs, token limits or a Devin integration.

Branch promotion and standards

Evidence: S02, S11 and S12

The canonical path is feature branch to weekly-integration through peer review, then promotion to dev. dev is the stable default line and replaces the former master path. Feature work must not merge directly into dev.

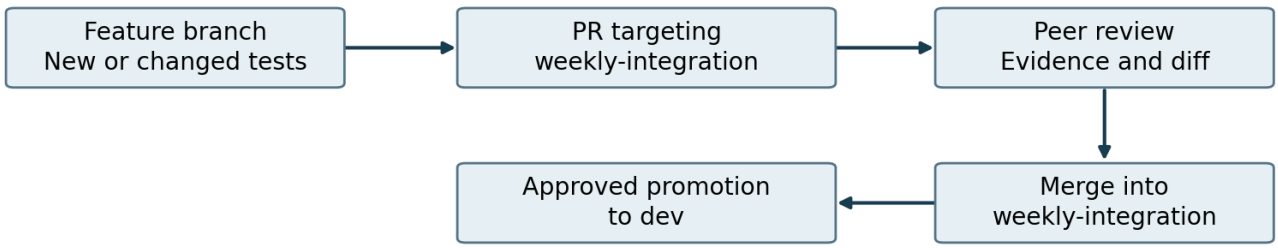


Figure 6. Documented branch path. QA lead responsibilities include reviewing feature PRs and approving promotion to dev.

Work type	README naming pattern
Feature or new test	feat/... or test/<owner>-<test-id>
Batch of tests	test/<owner>-<short-desc>
Bug fix or refactor	fix/<owner>-<description>
Infrastructure, config or docs	docs/... or config/<owner>-...

Standards that must be consulted

CODING_STANDARDS.md defines the framework contract. AGENTS.md and .github/control-plane/README.md define agent entry and routing rules. .github/CONTRIBUTING.md and the pull request template guide contribution and promotion. Their full contents are not reproduced here because only references to them are visible.

Recommendation: reviewers should verify app fixture imports, file placement, meaningful assertions, targeted test evidence, protected platform skill boundaries and absence of secrets. Actual branch protection settings and required CI checks require repository configuration evidence.

Security and operational reliability

Evidence: S03, S10 to S13; recommendations clearly marked

Documented requirements

- Use the FPL network and approved test credentials for local setup.
- Keep the QMetry API key in local MCP configuration and do not commit .vscode/mcp.json.
- Route lifecycle work before editing POMs or invoking app lifecycle tools.
- Do not change shared .github/skills/ for application behavior.
- Follow reviewed promotion through weekly-integration to dev.

Recommended controls to verify

Risk	Recommended check	Evidence required
Credential leakage	Keep .env and secret bearing config out of commits; apply approved secret management.	Ignore rules, secret scanning and CI secret references.
Wrong target or destructive action	Use least privilege test accounts and explicitly approved environment and data.	Profiles, role definitions and test data policy.
Sensitive reports	Restrict access to screenshots, traces and exports; define retention.	Artifact permissions, redaction rules and retention settings.
Unreviewed AI changes	Require diff review and actual test evidence; preserve assertions.	PR checks, review policy and run links.
Dependency drift	Compare lockfile and runtime; record separately resolved MCP versions.	package files, CI configuration and tool logs.
Flaky or misleading tests	Separate product defects from test and environment failures.	Failure trends, retries policy and triage records.

Do not mistake recommendations for configured controls

The screenshots do not demonstrate role based access enforcement, secret scanning, approval implementation, retry policy, observability infrastructure, or compliance certification. The table is a review agenda for the project owner, not a list of controls proven to be active.

Safe handling during diagnosis

Preserve the failing run before changing tests. Avoid sharing raw credentials or sensitive application data. Treat downloaded workbooks, traces and browser sessions as potentially sensitive. Stop when access or expected behavior is unclear rather than expanding permissions or weakening validation.

Troubleshooting and project handoff

Evidence: Derived guidance from S03 to S13

The README table of contents names troubleshooting and a full local setup guide, but their detailed contents are not visible. The checks below are recommendations based on the documented architecture, not a transcription of that missing guide.

Symptom	First checks
A command is not recognized	Check the shell and installed tools. Use PowerShell for $env: syntax; reopen a terminal after approved PATH changes.
No tests are discovered	Check APP_ID, file argument or --grep/--tags, current branch and app readiness caveats. --list does not execute a test.
Wrong test subset runs	Do not rely on SUITE_ID. Inspect the actual Playwright pattern or Cucumber tag expression.
Browser cannot start	Confirm npm dependencies and the matching Playwright browser installation.
Authentication or navigation fails	Check authorized network, TEST_ENV, test account and app configuration before editing assertions.
Generated test violates structure	Check router assignment, app fixture imports, POM path and app standards.
Report seems stale	Confirm run identity; allure:open opens the last generated report. Generate the intended report before reviewing.
QMetry result is missing	Check JUnit file, upload script, mapping and authorized credentials. Report creation alone does not prove upload success.

Recommended handoff package

- Purpose, app, test case identifiers and expected behavior.
- Branch and revision; files changed and lifecycle phase used.
- Exact commands, filters, environment and resolved tool versions.
- Results and report locations; relevant traces and screenshots.
- Known failures, readiness blockers and follow up owner.
- PR target weekly-integration, reviewer decisions and promotion status.

A useful handoff enables another engineer to reproduce the scope without guessing. Do not put secrets in the handoff, and do not claim completion when only discovery, code generation or a local configuration edit has occurred.

Evidence register and open questions

The reference consolidates 13 user supplied README screenshots for the repository identified as NextEraEnergy/yt213-agentx-automation. Source IDs below follow the supplied image order. They identify visible sections without reproducing photographs or incidental screen content.

Source	Visible README content
S01	Title, purpose, stack summary and table of contents.
S02	Project actions, user roles and quick start prerequisites.
S03	Quick start and beginning of single test commands.
S04	Single test examples and app readiness caveats.
S05	Suite execution, selection controls and report commands.
S06	Reports, JUnit naming and architecture introduction.
S07	Control plane layer diagram and entrypoint relationship.
S08	Lifecycle diagram and layer responsibilities.
S09	App context, precedence and runtime application map.
S10	AI tooling, MCP registration and project tree beginning.
S11	Repository structure and placement rules.
S12	Key rules, promotion pathway and branch names.
S13	Technology stack and documented versions.

What remains to verify against the repository

Obtain the current README and revision, package and lock files, Playwright/Cucumber configuration, app profiles, AGENTS.md, lifecycle skill contents, standards, pipeline YAML and upload scripts. Verify test inventory and app readiness, exact report naming, credentials handling, test data cleanup, retry and parallelism behavior, and access policies.

The screenshots are a partial documentation snapshot. They do not expose every README section or prove implementation behavior. No repository execution, network authentication, CI run, browser test, QMetry upload or credential validation was performed to create this reference.

Terms used

POM: page object model. Fixture: app scoped test setup or dependency support. MCP: Model Context Protocol tool interface. BDD: behavior driven development. TC: test case. KB: knowledge base. CI/CD: continuous integration and delivery. runStem: the report name stem, whose exact construction must be checked in project code.